Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО»

Факультет информационных технологий и программирования
Кафедра прикладной информатики

ЛАБОРАТОРНАЯ РАБОТА №2

Реализация микросервисов

Выполнил: студент группы К3341
Кижваткин Н.М.

Санкт-Петербург
2025

1. ЦЕЛЬ И ЗАДАЧИ

Цель работы: реализация микросервисной архитектуры системы Callmentor с разработкой четырёх основных сервисов и обеспечением их взаимодействия.

Задачи: реализовать сервис Gateway (API и бизнес-логика), ML-service (оркестратор ML-пайплайна), ml-asr-worker (транскрибация речи), ml-llm-worker (LLM-анализ). Обеспечить взаимодействие сервисов через HTTP и RabbitMQ. Реализовать систему наблюдаемости.

2. СЕРВИС GATEWAY

2.1. Структура и технологии

Gateway — центральный API-сервис системы, реализованный на FastAPI. Запускается на порту 8000. Использует asyncpg для асинхронной работы с PostgreSQL, Alembic для миграций, APScheduler для фоновых задач, python-jose для JWT, bcrypt для хэширования паролей, httpx для HTTP-клиента к ML-сервису, scalar-fastapi для документации.

2.2. Ключевые реализованные модули

PermissionMiddleware — middleware для проверки прав доступа на основе RBAC. Каждый входящий HTTP-запрос проверяется на наличие Bearer-токена и наличие права на данный маршрут в таблице m2m_role_routes.

DatabaseWorkerAsync — универсальный асинхронный слой работы с БД. Предоставляет методы custom_orm_select (гибкий SELECT с фильтрами, сортировкой, пагинацией), count, insert, update, batch_insert. Использует пул соединений SQLAlchemy.

DashboardService — сервис агрегации данных для дашборда. Реализует сложные SQL-запросы с JOIN, GROUP BY, оконными функциями для расчёта статистики по звонкам, менеджерам и стадиям.

BitrixDownloaderScheduler — фоновый загрузчик звонков из Bitrix24. Использует Bitrix REST API для получения списка звонков, фильтрации по менеджерам и периоду, скачивания аудиофайлов и их автоматической передачи в ML-пайплайн.

3. СЕРВИС ML-SERVICE (ОРКЕСТРАТОР)

3.1. Архитектура и технологии

ML-service реализован на FastAPI с использованием FastStream для работы с RabbitMQ. Является точкой входа для всех задач ML-обработки. Принимает аудиофайл от Gateway, анализирует его свойства и направляет в нужную очередь обработки.

3.2. Логика маршрутизации аудио

Функция run_pipeline выполняет следующий алгоритм: определяет количество каналов через pydub/AudioSegment; для стерео-файлов вычисляет корреляцию между левым и правым каналами с помощью numpy.corrcoef; если корреляция > 0.999 (каналы идентичны — "задвоение") → конвертирует в моно и направляет на диаризацию; если стерео с разными каналами → направляет напрямую в ASR_TRANSCRIPTION_QUEUE; если моно → направляет в ASR_DIARIZATION_QUEUE.

4. СЕРВИС ML-ASR-WORKER (ТРАНСКРИБАЦИЯ)

4.1. Архитектура и технологии

ml-asr-worker реализован на базе FastStream (надстройка над aio_pika для работы с RabbitMQ). Обрабатывает три типа очередей: ASR_TRANSCRIPTION_QUEUE (прямая транскрибация стерео), ASR_DIARIZATION_QUEUE (моно с предварительной диаризацией), ASR_MANUAL_REVIEW_QUEUE (parking-очередь ошибок). Использует библиотеки Torch, torchaudio, onnxruntime для ML-инференса.

4.2. Модель транскрибации

Транскрибация реализована в asr_core.py с поддержкой двух режимов:

1. VastAI (приоритетный): инференс через Triton Inference Server на GPU-машине VastAI. Используется клиент tritonclient.grpc. При недоступности Triton автоматически переключается на offline-режим.
2. Offline (fallback): локальная транскрибация через модель T-one (ToneTranscriber), реализованную командой VoiceKit. Использует onnxruntime и kenlm для декодирования.

StereoDiarizer (stereo_diarizer.py) реализует диаризацию стерео-аудио: разделение каналов и их поочерёдная транскрибация с сохранением информации о спикере (менеджер/клиент).

5. СЕРВИС ML-LLM-WORKER (АНАЛИЗ LLM)

5.1. Архитектура и технологии

ml-llm-worker реализован на FastStream. Использует библиотеку openai для взаимодействия с OpenAI API и Cloud.ru API. Дополнительно применяет sentence-transformers и scikit-learn для кластеризации возражений.

5.2. Реализованные обработчики очередей

_llm_audio_subscriber: основной обработчик LLM-задач. Принимает транскрипцию и набор промтов, вызывает соответствующий провайдер (openai или cloud_ru) через llm_core.process_transcription. Поддерживает retry-механизм через RetryableLLMError.

_kpi_subscriber: обработчик задач KPI-отчётов. Принимает список суммаризаций звонков менеджера и статистику промтов, генерирует текстовый KPI-отчёт через LLM (kpi_core.process_kpi_report).

_objections_subscriber: обработчик кластеризации возражений. Использует sentence-transformers для векторизации текстов возражений и алгоритмы кластеризации scikit-learn для выявления паттернов.

_postprocess_subscriber: постобработка результатов LLM-анализа. Форматирует и структурирует результаты перед отправкой колбэка.

5.3. Системные промты

Для каждого звонка дополнительно выполняются два обязательных системных промта: summary — суммаризация диалога (краткое содержание звонка) и recommendations — персональные рекомендации для менеджера с указанием конкретных фраз для улучшения. Оба промта настраиваются для каждой компании через таблицу system_prompts.

6. РЕАЛИЗАЦИЯ СИСТЕМЫ НАБЛЮДАЕМОСТИ

Все сервисы реализуют единый формат структурированного логирования: %(asctime)s - %(name)s - %(levelname)s - task_id=%(task_id)s - %(message)s. Поле task_id вставляется через кастомный logging.Filter (install_task_id_filter), что обеспечивает трассировку одной задачи через все сервисы по единому идентификатору.

Promtail подключён к Docker-сокету и собирает логи всех контейнеров, передавая их в Loki. Grafana предоставляет четыре преднастроенных дашборда для мониторинга.

7. ВЫВОДЫ

Реализованы четыре основных микросервиса системы Callmentor. Gateway предоставляет полноценный REST API с JWT-аутентификацией и ролевым доступом. ML-service координирует ML-пайплайн. ml-asr-worker выполняет транскрибацию с поддержкой GPU-инференса и fallback на CPU. ml-llm-worker реализует LLM-анализ через два провайдера. Система наблюдаемости обеспечивает сквозную трассировку задач и мониторинг состояния.